

Compile Principles: Lab02

2026-01-09

Aksel Shen

20234001053@m.scnu.edu.cn

South China Normal University

1 Introduction

A parser is a critical component of a compiler that performs syntactic analysis on a sequence of tokens produced by the lexer. It verifies that the token sequence conforms to the grammar rules of the programming language and builds an Abstract Syntax Tree (AST) that represents the program's structure.

In this lab, we implemented a parser for Tiny++, an extended version of the Tiny language that includes while loops, for loops, additional arithmetic operators (`%`, `^`), compound assignment operators (`+=`, `-=`), and extended comparison operators (`>`, `>=`, `<=`, `<>`).

2 Implementation

2.1 Lexical Analysis

We used the [Logos](#) crate to implement the lexer for Tiny++. The lexer tokenizes the input source code into a sequence of tokens, including keywords (`if`, `while`, `for`, `read`, `write`), identifiers, integer constants, operators, and punctuation symbols.

Comments enclosed in curly braces (`{...}`) and whitespace are automatically skipped during tokenization.

2.2 Parsing Strategy

We implemented a recursive descent parser, which is a top-down parsing method that constructs the parse tree starting from the root and proceeding towards the leaves. Each non-terminal in the grammar corresponds to a parsing function.

The parser maintains the current token position and provides helper methods to match expected tokens, consume tokens, and report syntax errors.

2.3 Grammar Support

Our parser supports the complete Tiny++ grammar, including:

Statements:

- `if-then-end` and `if-then-else-end` conditionals
- `while-do-enddo` loops
- `for-to-enddo` and `for-downto-enddo` loops
- `repeat-until` loops
- `read` and `write` I/O statements
- Assignment statements with `:=`, `+=`, and `-=` operators

Expressions:

- Comparison operators: `<`, `=`, `>`, `<=`, `>=`, `<>`
- Arithmetic operators: `+`, `-`, `*`, `/`, `%`, `^`
- Proper operator precedence and associativity
- Parenthesized expressions

2.4 AST Construction

The parser constructs an AST represented by Rust enumerations and structures. The main AST node types include:

- Program: Root node containing a sequence of statements
- Statement: Represents various statement types (assignment, control flow, I/O)
- Expression: Represents expressions with operators and operands
- BinaryOp and AssignOp: Operator types

The AST provides a structured representation of the program that can be easily traversed for semantic analysis, optimization, or code generation.

3 Testing

We developed a comprehensive test suite with 11 test cases covering various language features. Each test case is a .tpp file that exercises specific aspects of the Tiny++ language.

Test cases include:

- Basic arithmetic and assignments
- Conditional statements (if-else)
- Loop constructs (while, for, repeat-until)
- Operator precedence and associativity
- Nested control structures
- Complete programs (factorial, GCD)

The test suite can be executed using shell scripts that automate the testing process, verifying that the parser correctly handles valid programs and produces appropriate AST representations.

4 Conclusion

In this lab, we successfully implemented a parser for the Tiny++ language using recursive descent parsing in Rust. The parser correctly handles the extended grammar, including loops, operators, and control flow statements.

The implementation demonstrates the practical application of parsing theory, including grammar rules, operator precedence, and AST construction. The use of Rust and the Logos library provided a robust and efficient foundation for building the parser.

This experience has deepened our understanding of syntactic analysis and the structure of compilers, preparing us for subsequent phases of compiler construction such as semantic analysis and code generation.